

Guia de Estudio — Tema 1

Fundamentos de IA Generativa

Modelos generativos aplicados al analisis de datos

PSU — IA Generativa Aplicada al Analisis de Datos | UNIR

Indice de contenidos

- 1. Conceptos fundamentales de IA generativa
- 2. Hitos historicos clave
- 3. Modelos generativos vs. modelos predictivos (discriminativos)
- 4. Arquitecturas principales
 - 4.1 Redes Generativas Antagonicas (GAN)
 - 4.2 Autoencoders Variacionales (VAE)
 - 4.3 Modelos de Difusion
 - 4.4 Transformers y modelos de lenguaje (LLM)
- 5. Aplicaciones en analisis de datos
- 6. Metricas de evaluacion
- 7. Limitaciones y desafios
- 8. Resumen comparativo de arquitecturas
- 9. Preguntas de autoevaluacion

1. Conceptos fundamentales de IA generativa

Definicion: IA generativa

Rama de la inteligencia artificial enfocada en el desarrollo de sistemas capaces de crear contenido original, novedoso y coherente. Estos modelos aprenden las *distribuciones subyacentes* de los datos de entrenamiento para generar nuevas instancias que se asemejen a los datos originales.

El principio fundamental es el siguiente: dado un conjunto de datos de entrenamiento

$X = \{x_1, x_2, \dots, x_n\}$, un modelo generativo busca aprender la distribución de

probabilidad $p(x)$ que genere esos datos. Una vez aprendida, el modelo puede

muestrear nuevas instancias $x_{\text{new}} \sim p(x)$ que no existían en el conjunto original.

Matemáticamente, el objetivo se puede expresar como:

$$\hat{p}(x) \approx p_{\text{data}}(x)$$

donde $\hat{p}(x)$ es la distribución aprendida por el modelo y $p_{\text{data}}(x)$ es la

distribución real (desconocida) de los datos.

Intuición clave

Pensar en la IA generativa como un artista que estudia miles de obras y luego crea nuevas piezas originales que mantienen el "estilo" aprendido, sin copiar ninguna obra específica. El modelo no memoriza los datos: aprende la estructura estadística que los genera.

En el contexto del análisis de datos, esta capacidad se traduce en: generar conjuntos de datos sintéticos, simular escenarios complejos, automatizar la creación de informes y visualizaciones, e imputar valores faltantes.

2. Hitos históricos clave

Etapas	Periodo	Hitos	Relevancia
Fundamentos lingüísticos	1883–1916	Breál, Saussure	Principios estructurales y semánticos del lenguaje, base para PLN
Origen de la IA			

	1950–1960	Test de Turing, Perceptron, Machine Learning	Nace la IA como disciplina; se exploran conceptos de inteligencia y aprendizaje automatico
Primeros sistemas	1966–1980	ELIZA, PLN simbolico, modelos estadisticos IBM	Primeros intentos de interaccion lenguaje-maquina
Transformacion estadistica	1997–2013	LSTM, Word2Vec, CoreNLP	Transicion del enfoque simbolico al vectorial; mejora en memoria secuencial y representacion semantica
Revolucion de modelos	2014–2020	GAN, Transformer, BERT, GPT-2/3	Emergencia de modelos generativos altamente escalables
IA generativa multimodal	2023–2024	GPT-4, Gemini, Claude 3, Mistral Large	Integracion de multiples modalidades (texto, imagen, audio)

Conexion entre hitos

La evolucion sigue una cadena causal: los fundamentos linguisticos permitieron el PLN simbolico, que evoluciono hacia representaciones vectoriales (Word2Vec), las cuales habilitaron las arquitecturas de atencion (Transformers), que a su vez son la base de los LLM actuales.

3. Modelos generativos vs. modelos predictivos (discriminativos)

Definicion formal: Modelo discriminativo

Aprende la distribucion condicional $p(y | x)$, es decir, la probabilidad de una

etiqueta y dado un dato de entrada x . Responde a: "dada esta entrada, cual es la salida mas probable?"

Definicion formal: Modelo generativo

Aprende la distribucion conjunta $p(x, y)$ o directamente la distribucion marginal $p(x)$. Puede generar nuevas instancias de datos. Responde a: "como se ve un dato tipico de esta distribucion?"

La relacion entre ambos se establece mediante el **teorema de Bayes**:

$$p(y | x) = \frac{p(x | y) \cdot p(y)}{p(x)}$$

donde $p(x | y)$ es la verosimilitud (likelihood), $p(y)$ es la probabilidad a

priori (prior) y $p(x)$ es la evidencia. Un modelo generativo que aprende

$p(x | y)$ y $p(y)$ puede, en principio, derivar $p(y | x)$ y asi realizar

tareas discriminativas tambien.

Caracteristica	IA predictiva (discriminativa)	IA generativa
Funcion principal	Clasificacion y prediccion: estima $p(y x)$	Generacion: aprende $p(x)$ para crear contenido nuevo
Pregunta clave	"Que es probable que suceda?"	"Que contenido nuevo puedo crear que siga este patron?"
Tareas comunes	Regresion, clasificacion, forecasting, clustering	Generacion de texto, imagenes, datos sinteticos, simulacion
Salida tipica		

	Valores numericos, etiquetas, probabilidades	Textos, imagenes, audio, conjuntos de datos simulados
Valor en analisis	Identifica patrones, automatiza decisiones	Automatiza tareas creativas, genera datos sinteticos

Ejemplo practico

Discriminativo: dado un correo electronico, predecir si es spam

$$(p(y = \text{spam} \mid x)).$$

Generativo: aprender como "lucen" los correos spam y generar ejemplos sinteticos nuevos de correos spam para entrenar un modelo mas robusto.

4. Arquitecturas principales

4.1 Redes Generativas Antagonicas — GAN

Definicion: GAN (Generative Adversarial Network)

Arquitectura propuesta por Goodfellow et al. (2014) compuesta por dos redes neuronales

que compiten en un juego de suma cero: un **Generador** G y un **Discriminador**

D .

Componentes:

Generador $G(z)$: toma como entrada un vector aleatorio $z \sim \mathcal{N}(0, I)$ del

espacio latente y produce una muestra sintetica $G(z)$ que busca imitar los datos reales.

Discriminador $D(x)$: recibe muestras (reales o sinteticas) y produce una

probabilidad $D(x) \in [0, 1]$ que indica la verosimilitud de que x sea un dato real.

Funcion objetivo — Juego min-max:

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$

Desglose termino por termino:

$E_{x \sim p_{\text{data}}} [\log D(x)]$ — *Primer termino:* la esperanza sobre datos reales. El

discriminador quiere maximizar esto, es decir, asignar $D(x) \rightarrow 1$ a los datos reales.

Como $\log(1) = 0$ y $\log(0) \rightarrow -\infty$, maximizar este termino empuja a

D a reconocer correctamente los datos reales.

$E_{z \sim p_z} [\log(1 - D(G(z)))]$ — *Segundo termino:* la esperanza sobre datos

generados. El discriminador quiere maximizarlo haciendo que $D(G(z)) \rightarrow 0$

(rechazar los falsos), mientras que el generador quiere minimizar toda la expresion, es decir,

hacer que $D(G(z)) \rightarrow 1$ (engañar al discriminador).

Equilibrio de Nash (optimo teorico)

En el equilibrio, el generador ha aprendido perfectamente la distribucion de los datos reales ($p_G = p_{\text{data}}$), y el discriminador ya no puede distinguir entre datos

reales y sinteticos: $D^*(x) = \frac{1}{2}$ para todo x . Es decir, el discriminador responde "50-50" ante cualquier muestra.

Discriminador optimo: para un generador G fijo, el discriminador optimo es:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$$

Esta expresion dice que D asigna probabilidad proporcional a cuanto mas probable es que x venga de los datos reales que del generador.

Aplicacion en analisis de datos:

Aumentar conjuntos de datos escasos (especialmente clases minoritarias), proteger la privacidad generando datos sinteticos en lugar de compartir datos reales, y simular escenarios hipoteticos.

Problemas conocidos de las GAN

Colapso de modo (mode collapse): el generador produce solo unas pocas variantes en lugar de la diversidad completa de los datos reales. *Inestabilidad de entrenamiento:* el equilibrio entre G y D es delicado; si uno domina excesivamente, el entrenamiento diverge.

4.2 Autoencoders Variacionales — VAE

Definición: VAE (Variational Autoencoder)

Modelo generativo propuesto por Kingma y Welling (2014) que combina aprendizaje profundo con inferencia variacional bayesiana. Aprende una representación latente estructurada y continua de los datos, permitiendo generar nuevas muestras.

Componentes:

Codificador (Encoder) $q_\phi(z | x)$: dado un dato de entrada x , produce los parámetros de una distribución en el espacio latente, típicamente una gaussiana:

$$q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)) \quad .$$

Decodificador (Decoder) $p_\theta(x | z)$: dado un punto z del espacio latente, reconstruye el dato original x .

Espacio latente: es una representación comprimida y continua del conocimiento aprendido. A diferencia de un autoencoder clásico que codifica como un punto fijo, el VAE codifica como una distribución, lo que introduce aleatoriedad controlada y permite la generación.

Función objetivo — ELBO (Evidence Lower Bound):

$$\mathcal{L}(\theta, \phi; x) = E_{q_\phi(z | x)} [\log p_\theta(x | z)] - D_{KL}(q_\phi(z | x) || p(z))$$

Desglose término por término:

$$E_{q_\phi(z|x)} [\log p_\theta(x|z)] \quad \text{— } \textit{Termino de reconstruccion: mide que tan bien el}$$

decodificador reconstruye x a partir de z . Se quiere maximizar: que los datos reconstruidos se parezcan a los originales.

$$D_{KL}(q_\phi(z|x) || p(z)) \quad \text{— } \textit{Divergencia KL (regularizacion): mide la "distancia" entre}$$

la distribucion latente aprendida $q_\phi(z|x)$ y la distribucion a priori

$$p(z) = \mathcal{N}(0, I) \quad . \text{ Se quiere minimizar: que el espacio latente sea "suave" y siga una}$$

forma conocida, de modo que podamos muestrear nuevos z y generar datos.

La divergencia KL para gaussianas tiene forma cerrada:

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = -\frac{1}{2} \sum_{j=1}^J (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)$$

donde J es la dimension del espacio latente. Cada termino penaliza desviaciones de

$$\mu_j \quad \text{respecto a 0 y de } \sigma_j^2 \quad \text{respecto a 1.}$$

Truco de reparametrización (reparameterization trick)

Para poder calcular gradientes a través del muestreo (que normalmente no es

diferenciable), se usa: $z = \mu + \sigma \cdot \epsilon$ donde $\epsilon \sim \mathcal{N}(0, I)$. Así, la

aleatoriedad queda en ϵ , y μ y σ son funciones diferenciables de los parámetros del codificador.

Por que se llama ELBO: ELBO significa *Evidence Lower Bound*, es decir, una cota inferior de la

log-evidencia $\log p(x)$. Formalmente:

$$\log p(x) \geq \mathcal{L}(\theta, \phi; x)$$

Maximizar el ELBO es una forma tratable de (aproximadamente) maximizar la verosimilitud

$p(x)$, que es intratable directamente.

Aplicación en análisis de datos: generación controlada de datos sintéticos, imputación de valores faltantes, reducción de dimensionalidad (el espacio latente actúa como PCA no lineal), detección de anomalías (puntos con baja probabilidad en el espacio latente).

Ventajas y limitaciones

Ventajas: entrenamiento estable (sin adversarial training), espacio latente interpretable y continuo, facilita la interpolación entre ejemplos.

Limitaciones: tiende a generar muestras "borrosas" o promedios, menor nitidez que las GAN.

4.3 Modelos de Difusion

Definicion: Modelos de difusion (Diffusion Models)

Modelos generativos que aprenden a invertir un proceso de ruido gradual. Operan en dos fases: un proceso *forward* que destruye la estructura de los datos agregando ruido paso a paso, y un proceso *reverse* que aprende a reconstruir los datos a partir de ruido puro.

Proceso forward (difusion): se agrega ruido gaussiano de forma incremental en T pasos:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

donde $\beta_t \in (0, 1)$ es el schedule de ruido que controla cuanto ruido se agrega en

cada paso. Despues de T pasos suficientemente grandes, $x_T \approx \mathcal{N}(0, I)$.

Gracias a la propiedad de composicion gaussiana, se puede saltar directamente al paso

t :

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

donde $\alpha_t = 1 - \beta_t$ y $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

Proceso reverse (denoising): el modelo aprende a invertir cada paso:

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \sigma_t^2 I)$$

En la practica, el modelo (usualmente una red U-Net) predice el ruido $\epsilon_{\theta}(x_t, t)$ que fue agregado, y la media se calcula como:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

Funcion de perdida simplificada:

$$L_{\text{simple}} = E_{t, x_0, \epsilon} [\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2]$$

Es decir, se entrena al modelo para que prediga el ruido ϵ que fue agregado en cada paso, minimizando el error cuadratico medio entre el ruido real y el predicho.

Aplicacion en analisis de datos: sintesis de datos tabulares complejos, generacion de series temporales con patrones realistas, aumento de datos en dominios donde la fidelidad a la distribucion real es critica.

4.4 Transformers y modelos de lenguaje (LLM)

Definicion: Transformer

Arquitectura introducida por Vaswani et al. (2017) basada enteramente en mecanismos de autoatencion (self-attention), sin recurrencia ni convolucion. Procesa todos los elementos de una secuencia en paralelo, capturando dependencias de largo alcance de manera eficiente.

Mecanismo de autoatencion (Self-Attention):

Para cada token de la secuencia, se computan tres vectores a partir de matrices de peso aprendidas:

$$Q = XW^Q \quad (\text{Query — "que busco"}), \quad K = XW^K \quad (\text{Key — "que ofrezco"}),$$

$$V = XW^V \quad (\text{Value — "que contenido tengo"})$$

La función de atención escalada es:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Desglose:

$$QK^T \quad \text{— producto punto entre queries y keys. Mide la "similitud" o "relevancia" entre}$$

cada par de tokens. Si el query del token i tiene alta similitud con la key del token

j , entonces i "prestara atención" a j .

$\sqrt{d_k}$ — factor de escalado, donde d_k es la dimensión de las keys. Sin este

factor, los productos punto crecen con la dimensión, lo que produce gradientes muy pequeños

después del softmax (saturación). Escalar por $\sqrt{d_k}$ mantiene la varianza estable.

$\text{softmax}(\cdot)$ — convierte los puntajes de similitud en pesos de atencion

normalizados (suman 1). Cada token tiene un vector de pesos que indica cuanto "atiende" a cada otro token.

$\cdot V$ — los pesos de atencion ponderan los vectores de valor, produciendo la salida: una representacion contextualizada de cada token.

Intuicion de la autoatencion

Es como un buscador que, para cada palabra de una oracion, decide cuanto le importa cada una de las demas palabras del contexto. En "El gato se sento en la alfombra porque tenia sueno", la palabra "tenia" necesita "atender" a "gato" para resolver la referencia.

Multi-Head Attention: en lugar de una sola funcion de atencion, se ejecutan h cabezas

en paralelo, cada una con sus propias matrices W_i^Q, W_i^K, W_i^V . Las salidas se concatenan y proyectan:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Cada cabeza puede capturar un tipo diferente de relacion: una cabeza para relaciones sintacticas, otra para semanticas, otra para posicionales, etc.

Autoatencion enmascarada (causal): en modelos generativos tipo GPT, se aplica una mascara que impide que el token i atienda a posiciones futuras $j > i$. Esto garantiza la propiedad autoregresiva: la prediccion de cada token depende solo de los anteriores.

Generacion autoregresiva — regla de la cadena:

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i \mid x_1, \dots, x_{i-1})$$

Los LLM tipo GPT implementan esta descomposicion: predicen el siguiente token condicionado a todos los anteriores, usando la autoatencion enmascarada para respetar la causalidad.

Aplicacion en analisis de datos: generacion de consultas (texto a SQL/codigo), resumen y analisis de datos textuales, narrativa analitica automatica, asistencia en preprocesamiento y limpieza, explicabilidad de modelos.

5. Aplicaciones en analisis de datos

Aplicacion	Funcion principal	Beneficios clave
Generacion de datos sinteticos	Crear datos artificiales coherentes con los reales	Proteccion de datos sensibles; entrenamiento sin datos reales
Aumento de datasets	Generar mas ejemplos para modelos de ML	Mejora del rendimiento en datos escasos o desbalanceados
Simulacion de escenarios	Producir datos hipoteticos para analisis de riesgo	Evaluacion de situaciones what-if
Preparacion y limpieza	Automatizar transformaciones y correcciones	Ahorro de tiempo; reduccion de errores manuales
Generacion de codigo (NL2SQL)	Traducir lenguaje natural a codigo estructurado	Democratizacion del analisis
Narrativa analitica (NLG)	Crear resúmenes y explicaciones automaticas	Mejora de comprension para usuarios no tecnicos

6. Metricas de evaluacion

6.1 Para imagenes

Inception Score (IS)

Mide la calidad y diversidad de imagenes generadas usando un modelo de clasificacion preentrenado (Inception). Se basa en dos propiedades: (1) imagenes de alta calidad

producen distribuciones condicionales $p(y | x)$ con baja entropia (el clasificador

esta "seguro"), (2) las imagenes deben ser diversas, con distribucion marginal

$p(y)$ de alta entropia.

$$IS = \exp(E_x[D_{KL}(p(y | x) || p(y))])$$

Un IS alto indica imagenes realistas y diversas.

Frechet Inception Distance (FID)

Compara las distribuciones de activaciones de imagenes reales y generadas en una capa del modelo Inception. Modela ambas como gaussianas multivariadas y calcula la distancia de Frechet:

$$FID = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

donde (μ_r, Σ_r) y (μ_g, Σ_g) son la media y covarianza de las

activaciones reales y generadas respectivamente. **Un FID bajo indica mayor similitud entre distribuciones**, es decir, imagenes generadas mas realistas.

6.2 Para texto

Perplejidad (Perplexity)

Mide que tan bien un modelo de lenguaje predice una secuencia de prueba. Se interpreta como el "numero efectivo de opciones" que el modelo considera en cada paso:

$$PPL = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log p(x_i | x_1, \dots, x_{i-1})\right)$$

Menor perplejidad = mejor modelo. Una perplejidad de 10 significa que, en promedio, el modelo esta tan "confundido" como si eligiera uniformemente entre 10 opciones.

BLEU (Bilingual Evaluation Understudy)

Mide la similitud entre un texto generado y uno de referencia contando n-gramas coincidentes. Incluye una penalizacion por brevedad para evitar que textos muy cortos tengan puntuaciones artificialmente altas. Valores entre 0 y 1 (o 0 a 100). Usado originalmente en traduccion automatica.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Similar a BLEU pero orientado al recall: mide cuantos n-gramas del texto de referencia aparecen en el texto generado. Especialmente util para evaluar resúmenes automaticos.

BERTScore

Calcula la similitud semantica entre textos usando embeddings de BERT. En lugar de comparar n-gramas exactos, compara representaciones vectoriales contextualizadas, capturando parafraseos y sinonimos. Correlaciona mejor con juicios humanos que BLEU/ROUGE.

6.3 Para datos tabulares

Las metricas para datos tabulares sinteticos se dividen en dos dimensiones complementarias:

Fidelidad estadística: estadísticas descriptivas (medias, varianzas), correlaciones entre variables, visualización con PCA/t-SNE, pruebas de Kolmogorov-Smirnov (distribuciones continuas), prueba chi-cuadrado (distribuciones categóricas).

Utilidad práctica: TSTR (Train on Synthetic, Test on Real): entrenar un modelo con datos sintéticos y evaluar su rendimiento sobre datos reales. TRTS (Train on Real, Test on Synthetic): mide si los datos sintéticos son consistentes con los patrones aprendidos de datos reales. Métricas predictivas (Accuracy, F1).

Privacidad: membership attack — evaluar si es posible determinar si un registro específico del dataset original fue usado para entrenar el modelo generativo.

Tipo de dato	Métricas clave	Interpretación
Imágenes	IS, FID, Precision/Recall	IS alto y FID bajo = imágenes realistas y diversas
Texto	Perplejidad, BLEU, ROUGE, BERTScore	Métricas altas = mejor alineación con lenguaje natural
Datos tabulares	Estadísticas, correlaciones, PCA/t-SNE, TSTR/TRTS	Similitud estadística y rendimiento comparable = buena calidad

7. Limitaciones y desafíos

Limitación	Descripción	Impacto
Sesgos (bias amplification)	Los modelos replican y amplifican sesgos de los datos de entrenamiento	Datos sintéticos que perpetúan discriminaciones
Falta de explicabilidad	Funcionan como "cajas negras", dificultando la auditabilidad	Obstáculo para la confianza en entornos regulados
Riesgos de privacidad	Pueden "memorizar" datos sensibles del entrenamiento	Fuga de información si los sintéticos no están anonimizados
Uso indebido	Generación de deepfakes, datos fabricados, desinformación	Manipulación de mercados, fabricación de evidencia

Costos computacionales	Requieren GPU/TPU significativos para entrenamiento e inferencia	Barrera para organizaciones pequeñas
Alucinaciones	Producen contenido incoherente o factualmente incorrecto	Riesgo critico en aplicaciones de datos sensibles
Evaluacion compleja	No hay una unica metrica que capture todos los aspectos de calidad	Requiere combinacion de metricas automaticas y evaluacion humana

8. Resumen comparativo de arquitecturas

Aspecto	GAN	VAE	Difusion	Transformer/LLM
Mecanismo central	Juego adversarial (G vs D)	Codificacion-decodificacion variacional	Denoising iterativo	Autoatencion + generacion autoregresiva
Funcion de perdida	Min-max adversarial	ELBO (reconstruccion + KL)	MSE de prediccion de ruido	Cross-entropy del siguiente token
Espacio latente	Implicito (no estructurado)	Explicito, continuo, interpretable	Cadena de estados ruidosos	No explicitamente definido
Fortaleza clave	Alta nitidez y realismo	Estabilidad, interpretabilidad	Alta calidad, flexibilidad	Comprension contextual, flexibilidad
Debilidad clave	Inestabilidad, mode collapse	Muestras borrosas	Lentitud en generacion	Alucinaciones, costo computacional
Aplicacion tipica en datos	Datos sinteticos tabulares, aumento de clases	Imputacion, exploracion latente	Series temporales, datos complejos	NL2SQL, narrativa, generacion de codigo

9. Preguntas de autoevaluacion

Preguntas conceptuales

1.

Cual es la diferencia fundamental entre un modelo generativo y uno discriminativo, expresada en terminos de las distribuciones de probabilidad que aprenden?

2.

En la funcion objetivo de las GAN, explique que busca maximizar el discriminador y que busca minimizar el generador. Por que esto constituye un juego min-max?

3.

En el ELBO del VAE, que sucederia si eliminamos el termino de divergencia KL? Que problema practico generaria esto?

4.

Explique por que el factor de escalado $\frac{1}{\sqrt{d_k}}$ es necesario en la funcion de atencion del Transformer.

5.

Que es el colapso de modo (mode collapse) en las GAN y que estrategia propone CTGAN para mitigarlo?

6.

Interprete un FID de 15 vs uno de 120. Cual indica mejor calidad de generacion y por que?

7.

Por que se dice que la perplejidad mide el "numero efectivo de opciones" de un modelo de lenguaje? Demuestre la intuicion con un ejemplo de perplejidad = 5.

8.

Cual es la diferencia entre TSTR y TRTS, y por que ambos son necesarios para evaluar datos sinteticos tabulares?

9.

Nombre tres limitaciones eticas de la IA generativa y explique como impactan al analisis de datos.

10.

Compare las ventajas y desventajas de GAN vs. VAE para la generacion de datos tabulares sinteticos. En que escenario elegiria cada uno?

Preguntas matematicas

11.

Dado el discriminador optimo $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$, demuestre que

cuando $p_G = p_{\text{data}}$, entonces $D^*(x) = \frac{1}{2}$.

12.

En un VAE con espacio latente de dimension $J = 2$, si el codificador produce

$\mu = (0.5, -0.3)$ y $\sigma^2 = (1.2, 0.8)$, calcule la divergencia KL

con respecto a $\mathcal{N}(0, I)$.

13.

Explique la relacion entre la log-evidencia $\log p(x)$, el ELBO y la divergencia KL entre el posterior aproximado y el posterior real.